

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `if / elif / else` 的真值判定规则,与 C 的"非零即真"对比找差异
- 学会 `match` 语句(Python 3.10+)的结构匹配:值匹配、解构、守卫,并理解它为什么不是 C 的 `switch`
- 彻底告别 `for(i=0; i<n; i++)`,改用 `for x in iterable` 遍历 `list/dict/str/文件`,并掌握 `range / enumerate / zip` 三件套
- 掌握 `while` 的用法与 Python 没有 `do-while` 的替代写法
- 掌握 `break / continue` 与 Python 独门的 `for-else / while-else`
- 理解"没有 `goto`"的世界如何用提前返回与异常替代
- 写出符合 Pythonic 风格的循环:列表推导、生成器、`for-else` 查找惯用法

💡 从 C 到 Python:本章主题如何迁移

C 的流程控制由 `if/else`、`switch`、`for`、`while`、`goto` 统治;Python 对应 `if/elif/else`、`match` (3.10+)、`for...in`、`while`,**没有** `goto`,也没有 `do-while`。表面是语法换皮,底层是两种哲学。

第一,**条件判定从"非零"变成"真值"**。C 的 `if (x)` 问"x 是否非零";Python 的 `if x:` 问真值——`0`、`None`、空字符串、空列表、空字典、`False` 都是假,其余为真。这让代码更简洁(`if not items:` 直接判空),但也要求记住"哪些对象是假的",否则会写出 `if not scores:` 误杀"成绩为 0"的 bug(第2章预告过)。

第二,**循环从"计数"变成"遍历"**。C 的 `for(i=0; i<n; i++)` 是"用下标访问数组";Python 的 `for x in xs` 是"让迭代器逐个交出元素"。下标、边界、步长被迭代器协议封装(第6章细讲);代价是 `i += 2` 式跳跃遍历要显式写 `range(0, n, 2)`。

第三,**switch 进化成结构匹配**。C 的 `switch` 只能匹配整型常量、还有 `fallthrough` 陷阱;Python 的 `match` 能匹配任意对象、解构元组/列表/字典、带守卫条件,与 Rust 的 `match` 同源。它是新范式,初学先用它的 80% 能力(值匹配 + 解构)。

最后是 `goto`:C 用它做错误处理与跳出多重循环;Python 用 `try/except` (第9章)与"提前 `return` + 封装函数"替代。写 Python 时请忘掉 `goto`——你需要的一切都有更清晰的替代。

📦 前置知识自查(你已会的 C 基础)

- `if/else if/else` 的嵌套与悬挂 `else` 问题
- `switch` 的 `fallthrough` 行为与 `break` 的必要性
- `for(i=0; i<n; i++)` 三表达式循环与边界条件
- `while` 循环与 `do-while` 的"先执行后判断"语义
- `break / continue`; `goto` 的错误处理模式

🚀 本章学习路线

1. **第一阶段(约 1 小时,分支)**:读 §3.1–§3.2,把真值表和 match 的几种模式在 REPL 里各敲一遍
2. **第二阶段(约 1 小时,循环)**:读 §3.3–§3.4,练习 range/enumerate/zip 组合遍历
3. **第三阶段(约 1 小时,控制转移)**:读 §3.5–§3.6,做查找类的 for-else 练习
4. **第四阶段(实验,约 1.5 小时)**:完成章末实验"菜单驱动系统 + 素数筛选",覆盖全章
5. **验收标准**:不看资料把 C 的 `for(i=0;i<n;i++) printf("%d",a[i]);` 改写为 Pythonic 版本,并能说出 match 与 switch 的三个差异

本章知识导图

- §3.1 if/elif/else 与真值判定(真值表、悬挂 else、if 表达式)
- §3.2 match 结构匹配(值匹配、通配、解构、守卫)
- §3.3 for 循环:遍历万物(range/enumerate/zip、可迭代对象)
- §3.4 while 循环与 do-while 的替代写法
- §3.5 break/continue/else:循环的"完成标志"
- §3.6 没有 goto:提前返回、标志位、异常替代
- §3.7 循环惯用法:推导式、生成器、无限循环对照

§3.1 if/elif/else 与真值判定

语法上 Python 的 if 与 C 有三个差异:①条件后是**冒号**且不写括号;②分支体靠**缩进**而非花括号;③多分支用 `elif`。缩进即块——初学时会被吐槽,但它消灭了 C 里花括号错位引发的一整类 bug,团队里也不再有"这个 else 属于哪个 if"的争论。

C 的写法	Python 的写法
<code>if (x > 0) { ... }</code>	<code>if x > 0: + 缩进块</code>
<code>else if (y > 0)</code>	<code>elif y > 0:</code>
<code>if (ptr) /* 非空指针 */</code>	<code>if ptr is not None:</code>
<code>if (arr_len > 0)</code>	<code>if items: (非空容器为真)</code>
<code>if (flag == 0)</code>	<code>if not flag:</code>

3.1.1 真值判定表

下面这张表决定了一切 `if` 的行为,背熟它比背语法重要得多:

为假(False 值)	为真(True 值)
False 、 None	True
0、0.0、0j	任何非零数
空字符串 ""	任何非空字符串
空容器: []、()、{}、set()	任何非空容器
自定义对象的 __bool__ 返回 False	其余一切对象

示例 3-1 if/elif/else 与真值

```
def grade_of(score: float) -> str:
    if score >= 90:          # 条件后冒号, 不写括号
        return "优"
    elif score >= 80:        # elif 取代 C 的 else if
        return "良"
    elif score >= 60:
        return "及格"
    else:
        return "不及格"

# 真值判定: 判空容器直接 if
items = []
if items:                  # 空列表为假
    print("有内容")
else:
    print("空列表")

name = input("姓名: ").strip()
if not name:              # 空输入为假 → not 为真
    print("姓名不能为空")
```

⚠ 误区 1: 把 C 的“非零即真”直觉带进 Python

C 里 `if (x)` 只问“x 是不是 0”; Python 问“x 的真值”。两个经典翻车: ① `if not scores:` 想表达“没有成绩”, 但 `scores = [0]` (一份 0 分) 时列表非空, 为真, 走不进分支——这其实是对的, 但很多 C 程序员会误写成 `if scores == []:` (可以但啰嗦); ② 判断“是否 None”必须写 `if x is None:`, 写 `if x:` 会把 0、空串一起判掉。规则: 判 None 用 `is None`, 判空用 `not`, 判零用 `== 0`。

§3.2 match 结构匹配(Python 3.10+)

`match` 语句在 3.10 引入, 基本语法:

示例 3-2 match 值匹配

```
def describe(status: str) -> str:
    match status:
        case "ready":
            return "就绪"
        case "running":
            return "运行中"
        case "done":
            return "已完成"
        case _:
            # 通配:等价于 default
            return f"未知状态: {status!r}"
```

```
print(describe("ready"))      # 就绪
print(describe("crashed"))    # 未知状态: 'crashed'
```

match 是"语句"不是表达式;C 的 switch 要求整型常量,
match 可以匹配任意对象与类型!

3.2.1 解构匹配与守卫

match 的真正威力在于**解构**:把"形状"写出来,Python 自动拆包并匹配。配合 `if` 守卫(guard)可以做条件分支。它是处理"按类型/结构分派"的正规军,替代 C 里"switch 枚举 + 手写结构体解析"的组合。

示例 3-3 match 解构与守卫

```
def handle(event):
    match event:
        case ("click", x, y):
            # 元组解构
            print(f"点击位置 ({x}, {y})")
        case ["move", *path]:
            # 列表解构:首元素 + 其余
            print(f"移动路径: {path}")
        case {"type": "login", "user": u}:
            # 字典解构
            print(f"用户 {u} 登录")
        case {"type": "input", "value": v} if len(v) > 0:
            print(f"输入: {v}")
            # 守卫:if 进一步约束
        case _:
            # 通配兜底
            print(f"未知事件: {event!r}")

handle(("click", 10, 20))      # 点击位置 (10, 20)
handle(["move", (0, 0), (5, 5)]) # 移动路径: [(0, 0), (5, 5)]
handle({"type": "login", "user": "alice"}) # 用户 alice 登录
handle({"type": "input", "value": "hi"})    # 输入: hi
```

⚠ 误区 2:把 match 当 switch,问"为什么 case 里不能放表达式"

C 的 `case 1+1:` 是常量表达式;Python 的 `case` 是**模式**——写 `case x:` 会无条件匹配并把 `x` 绑定为被匹配对象(等于通配!),这是新手最懵的行为。想匹配"计算出来的值",用守卫: `case _ if v == threshold:`。另外 `case 1 | 2:` 表示"1 或 2"(或模式),不是位或。3.10 之前的旧代码库没有 match,仍用 if/elif 链,新项目放心用。

§3.3 for 循环:遍历万物

Python 的 `for` 只能遍历可迭代对象(list、str、dict、set、文件、range、生成器.....)。C 的三种 for 用法里,"遍历数组"和"遍历字符串"被统一成 `for x in xs` ;"计数循环"交给 `range()` 。下表是本章最实用的对照:

C 的写法	Python 的写法
<code>for (i=0; i<n; i++) printf("%d", a[i]);</code>	<code>for v in a: (直接遍历值)</code>
<code>for (i=0; i<n; i++) printf("%d", a[i]); (需要下标)</code>	<code>for i, v in enumerate(a):</code>
<code>for (i=0; i<n; i+=2)</code>	<code>for i in range(0, n, 2):</code>
<code>for (i=n-1; i>=0; i--)</code>	<code>for i in range(n-1, -1, -1): 或 for v in reversed(a):</code>
遍历两个数组	<code>for x, y in zip(a, b):</code>

3.3.1 range / enumerate / zip 三件套

这三个函数覆盖了 C 程序员在 for 循环里 90% 的下标操作:`range(start, stop, step)` 生成整数序列(左闭右开); `enumerate(xs)` 边遍历边给下标; `zip(a, b)` 并行遍历多个序列,长度不等时按最短的截断(3.10+ 加 `strict=True` 可在长度不等时抛错)。它们都返回"惰性"的迭代器而不是真列表,内存占用恒定——这是与 C 的数组下标循环在资源模型上的又一差异。

示例 3-4 range/enumerate/zip

```
# range:左闭右开,步长可正可负
print(list(range(5)))           # [0, 1, 2, 3, 4]
print(list(range(2, 10, 3)))    # [2, 5, 8]
print(list(range(10, 0, -2)))   # [10, 8, 6, 4, 2]

# enumerate:需要下标时
names = ["张三", "李四", "王五"]
for i, name in enumerate(names, start=1): # 从 1 开始编号
    print(f"{i}. {name}")

# zip:并行遍历
subjects = ["语文", "数学", "英语"]
scores = [88, 95, 91]
for subj, score in zip(subjects, scores):
    print(f"{subj}: {score}")

# zip(strict=True)(3.10+):长度不一立刻报错,防静默截断
# for x, y in zip([1, 2, 3], [10, 20], strict=True):
#     pass                      # ValueError: zip() argument 2 is shorter

# 转字典:zip 两序列合成 dict,一行搞定
grade = dict(zip(subjects, scores))
print(grade)                   # {'语文': 88, '数学': 95, '英语': 91}
```

3.3.2 遍历 dict / set / 文件

示例 3-5 遍历各种容器

```
# 遍历 dict:默认遍历键;键值用 .items()
student = {"name": "韩梅梅", "age": 18, "active": True}
for key in student:
    print(key)                # name age active
for key, value in student.items():
    print(f"{key} = {value}")

# 遍历 set:顺序无保证
tags = {"py", "c", "py", "go"}
for tag in tags:
    print(tag)                # 无序输出

# 遍历字符串:逐字符
for ch in "Python":
    print(ch.upper(), end=" ") # P Y T H O N

# 遍历文件:按行流式,大文件也安全(第7章细讲)
with open("data.txt", encoding="utf-8") as fh:
    for line in fh:           # 文件本身可迭代!
        print(line.rstrip())
```

⚠ 误区 3:遍历时修改容器(迭代器失效)

C++ 里遍历 vector 时 erase 会迭代器失效;Python 同样:遍历 list 时 remove / append 会跳过元素或报 RuntimeError。正确姿势是**遍历副本**: for x in lst[:] 或 for x in list(lst);筛选新集合用列表推导 [x for x in lst if ok(x)] 或原地重写: lst[:] = [x for x in lst if ok(x)]。删除 dict 的键也一样: for k in list(d): 再删。

§3.4 while 循环与 do-while 的替代

while 与 C 完全同义:条件为真就继续。Python **没有** do-while (先执行一次再判断)——这是有意为之,因为 while True: + 末尾 break 表达力相同且意图更清晰。输入校验循环是经典场景:

示例 3-6 while 与输入校验

```

# do-while 的替代:先执行,再判断
def read_age() -> int:
    while True:                # 模拟 do-while 的骨架
        raw = input("年龄: ").strip()
        try:
            age = int(raw)
            if 0 < age < 150:    # 链式比较
                return age
            print("年龄须在 0~150 之间")
        except ValueError:
            print("请输入整数")

n = 10
while n > 0:
    n -= 2                    # 没有 n--, 写 n -= 2
    print(n, end=" ")        # 8 6 4 2 0

# 哨兵循环:读到特定值退出(walrus, 见第2章)
while (line := input("> ").strip()) != "quit":
    print(f"收到: {line}")

```

§3.5 break / continue / else

`break` 立即跳出本层循环; `continue` 跳到下一次迭代——与 C 一致。Python 的独门武器是 `for-else / while-else` : 循环**没有被 `break` 打断而自然结束**时执行 `else` 块。它专治"查找类"问题:C 里要设 `found` 标志位:

示例 3-7 for-else:查找惯用法

```

# 场景:找第一个满足条件的元素,找不到要报告
def find_prime(limit: int) -> int:
    """返回大于 limit 的最小素数;不存在时打印提示"""
    for n in range(limit + 1, limit + 1000):
        for d in range(2, int(n ** 0.5) + 1):
            if n % d == 0:
                break          # 内层 break:不是素数
        else:                  # 内层没被 break → n 是素数
            return n
    else:                      # 外层跑完没 return(实际到不了)
        print("范围内没有素数")
    return -1

print(find_prime(100))        # 101
print(find_prime(90))         # 97

# 等价于 C 的标志位写法:found=0; for(...){...; found=1; break;}
# for-else 就是内置了这个标志位

```

🚀 工程建议:for-else 别炫技,见注释三分

for-else 是 Python 独有的语法,C 程序员第一次见一定懵。团队协作时,写上"# 循环完整跑完才进 else"这类注释,或者干脆封装成函数 + 提前 return(上面的 `find_prime` 就是好例子:else 只在表达"没找到"时出现一次)。生产代码的黄金法则是 循环体越短越好、退出路径越显式越好。

§3.6 没有 goto 的世界

Python 没有 `goto`。C 里 `goto` 的三个经典用途,Python 各有更清晰的替代:

C 的 goto 用法	Python 的替代
错误处理: <code>goto cleanup;</code>	<code>try/except/finally</code> (第9章)+ <code>with</code> (第5章)
跳出多层嵌套循环	封装成函数, <code>return</code> 直接退出;或标志位
重试逻辑	<code>while True:</code> + <code>continue / break</code>

示例 3-8 跳出多层循环的正规姿势

```
# 需求:在二维表里找第一个负数
table = [
    [1, 2, -3],
    [4, 5, 6],
    [7, 8, 9],
]

# 姿势一:函数 + 提前 return(最推荐)
def find_neg(mat) -> tuple | None:
    for i, row in enumerate(mat):
        for j, v in enumerate(row):
            if v < 0:
                return (i, j)      # 多层循环一次跳出
    return None                    # 找不到

print(find_neg(table))            # (0, 2)
```

§3.7 循环惯用法:推导式与生成器

列表推导 [表达式 for 变量 in 可迭代对象 if 条件] 是 Pythonic 风格的代表作:C 里"建新数组 + for 循环 + if 判断 + append"四步,推导式一行完成。它读起来像数学集合记号 $\{x^2 \mid x \in S, x > 0\}$ 。注意它**返回新列表**(副本),不修改原对象——所以它天然规避了遍历修改的坑。

示例 3-9 列表推导与生成器


```

nums = [1, 2, 3, 4, 5, 6]

# C 风格(反例,别这么写):
squares = []
for n in nums:
    if n % 2 == 0:
        squares.append(n * n)

# Pythonic:一行
squares = [n * n for n in nums if n % 2 == 0]
print(squares)                # [4, 16, 36]

# 字典推导 / 集合推导
square_map = {n: n * n for n in nums}
print(square_map[3])          # 9

# 生成器表达式:不一次性建列表,内存友好
total = sum(n * n for n in range(1_000_000)) # 括号内不建列表
print(total)                   # 333332833333500000

# 双层循环推导:拍平二维结构(先外层后内层)
matrix = [[1, 2], [3, 4]]
print([v for row in matrix for v in row])    # [1, 2, 3, 4]

```

⚠ 误区 4:推导式里写复杂逻辑

推导式适合"一个表达式 + 一个条件"。一旦出现多个 if、嵌套循环超过两层、或需要 `if/else` 分支,就拆回普通循环。判断标准:别人读你代码时,能 3 秒内说出推导式在干什么吗?不能,就拆。这是可读性优先于简洁的工程原则,Python 官方社区对"过度推导"同样持反对态度。

本章速查表

主题	写法 / 结论
if 语法	if 条件: 冒号 + 缩进; elif 多分支; else 兜底
真值	假值: 0、None、False、空容器、空串; if items: 判非空
判 None	if x is None: ,绝不写 if not x: 判 None
match	3.10+: case _: 通配; case 1 2: 或; if 守卫
计数循环	range(n)、range(a, b, step) 左闭右开
下标+值	for i, v in enumerate(xs, start=1):
并行遍历	for x, y in zip(a, b, strict=True):
倒序遍历	reversed(xs) 或 range(n-1, -1, -1)
遍历 dict	d.items() / d.values(); for k in d 是遍历键
遍历文件	for line in fh: 流式按行
do-while	无; 用 while True: + 末尾 break
break/continue	与 C 一致, 只影响本层循环
for-else	循环未被 break 时执行 else; 查找惯用法
goto	没有; try/except、return、标志位替代
推导式	[f(x) for x in xs if 条件]; 字典/集合推导
遍历修改	禁止; 遍历副本 xs[:] 或改用推导式

最小必会清单

- 能默写真值表: 6 种假值分别是什么
- 能写出 if / elif / else 完整结构, 并说明为什么不用 else if
- 能写出 match 的值匹配、通配 case _: 、守卫 if 、或模式 1 | 2
- 能解释 "match 的 case 写变量名是无条件匹配"
- 能把 for(i=0; i<n; i++) 三种变形(需要下标/步长/逆序)改写成 Python
- 能用 enumerate 带起始编号、zip 并行遍历、dict(zip(...)) 构造字典
- 能写出 do-while 的替代模式与哨兵循环(while (line := ...))
- 能写出 for-else 的查找惯用法, 并解释 else 何时执行
- 能说出 Python 里 "跳出多层循环" 的两种姿势
- 能写出列表推导与字典推导, 并判断何时该拆回普通循环
- 能说出遍历 list/dict 时修改容器的后果与正确做法

自测题

Q 1. 下列表达式哪些为真? `bool(0.0)`、`bool("0")`、`bool([])`、`bool([None])`、`bool({})`、`bool(0j)`。

✓ 参考答案

`False`、`True` (非空字符串,哪怕内容是字符 "0")、`False`、`True` (有一个元素的列表)、`False`、`False`。规则:空容器/零值/`None`/`False` 为假,其余为真。

Q 2. 用 `for-else` 写一个函数,判断整数 `n` 是否为素数(不用内建模块),并解释 `else` 的位置。

✓ 参考答案

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break          # 找到因子,跳出
    else:
        return True        # 完整跑完 → 没有因子 → 素数
    return False
```

`else` 挂在 `for` 上:循环没有被 `break` 打断时执行。注意 `range` 上限取到根号 `n` 即可,这是素数判定的经典优化。

Q 3. 写出与 C 代码等价的 Python: `for (i = 0; i < 10; i += 2) printf("%d ", i);`。

✓ 参考答案

`for i in range(0, 10, 2): print(i, end=" ")`, 输出 `0 2 4 6 8`。`range` 三参数分别对应起始、终止(不含)、步长。

Q 4. `match` 与 C 的 `switch` 有哪三个关键差异?

✓ 参考答案

① 匹配对象:`switch` 只能整型/枚举,`match` 可匹配任意对象、类型与结构;② 模式能力:`match` 支持解构、或模式 `|`、守卫 `if`;③ 无 `fallthrough`:每个 `case` 自动结束,不需要 `break`。另外 `case` 里写变量名是无条件绑定,不是常量比较。

Q 5. 用一行列表推导,从 `words = ["python", "c", "go", "java", "rust"]` 中选出长度大于 3 的单词,并转为大写。

✓ 参考答案

`[w.upper() for w in words if len(w) > 3]` → `['PYTHON', 'JAVA', 'RUST']`。结构:表达式 → `for` → `if`(过滤)。

Q 6. 遍历 `dict` 时删除部分键,直接 `for k in d: del d[k]` 会怎样?正确写法是什么?

✅ 参考答案

抛 `RuntimeError: dictionary changed size during iteration` (或行为不可靠)。正确写法:先取键副本——`for k in list(d):` 再删;或推导式重建 `d = {k: v for k, v in d.items() if 保留条件}`。列表同理,遍历 `lst[:]` 副本。

章末实验题:菜单驱动系统 + 素数筛选统计

📄 实验 3:文本菜单 + 素数筛选 + 数组遍历统计

实验目标:实现一个菜单驱动的命令程序,内置"素数筛选、数组统计、字符串分析"三个功能,强制用到 `if/elif`、`match`、三种循环、`break/continue`、`for-else`,并处理非法输入。

实验要求:

- 任务 1:主循环用 `while True` 打印菜单(1.素数筛选 2.数组统计 3.字符串分析 0.退出),用 `match` 分发功能(知识点:`while`、`match` 值匹配、通配)
- 任务 2:素数筛选功能:输入上限 `n`,用 `for...else` 找出 `2~n` 之间全部素数,每行输出 10 个(知识点:`for-else`、`range`、`break/continue`、格式化)
- 任务 3:数组统计功能:读入一串以逗号分隔的数字,统计个数、总和、平均值、最大值、最小值,负数与小数都要正确处理(知识点:类型转换、真值、内建函数 `sum/max/min`)
- 任务 4:字符串分析功能:输入一句话,统计其中字母、数字、空格、其他字符的个数,并用 `enumerate` 输出每个非空格字符的下标(知识点:遍历字符串、`enumerate`、真值)
- 任务 5:功能 3 用 `for i, v in enumerate(...)` 输出前三个元素的下标与值,展示"带下标的遍历"(知识点:`enumerate`)
- 任务 6:所有输入非法时给出提示并 `continue` 回主菜单;退出走 `break` 并用 `else` 输出"感谢使用"(知识点:`break/continue/while-else`)

实验环境:Python 3.10+(需要 `match`),单文件 `menu.py`。

第一步 写主循环骨架:`while True + match`,让 0 退出先通

第二步 素数筛选单独成函数,返回 `list`;输出时用 `enumerate` 或计数变量控制换行(每 10 个一行)

第三步 数组统计用 `split(",")` 拆分,列表推导 + `float()` 转换,非法数字用 `try/except` 捕获后返回空列表,主菜单根据"结果是否为空列表"判定是否打印统计(真值判定)

第四步 字符串分析用 `for ch in s:` 配合 `ch.isalpha()/ch.isdigit()/ch.isspace()`

✅ 参考答案要点(代码分两段)

参考实现约 100 行,分两段展示。第一段:素数筛选与数组统计。重点:for-else 判素数、推导式解析输入、enumerate 带编号输出、真值判定拦截空结果。

menu.py — 文本菜单驱动系统(覆盖第3章全部知识点)

```
def is_prime(n: int) -> bool:
    """for-else 判素数:循环没被 break 即为素数"""
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break
    else:
        return True
    return False

def find_primes(limit: int) -> list:
    """返回 2~limit 内全部素数"""
    return [n for n in range(2, limit + 1) if is_prime(n)]

def parse_numbers(text: str) -> list:
    """'1,2.5,-3' -> [1.0, 2.5, -3.0];含非法项返回空列表"""
    parts = [p.strip() for p in text.split(",") if p.strip()]
    nums = []
    for p in parts:
        try:
            nums.append(float(p))      # 类型转换 + 异常捕获
        except ValueError:
            return []                 # 非法输入,整体作废
    return nums

def menu_sieve() -> None:
    raw = input("请输入上限 n: ").strip()
    if not raw.isdigit():              # 真值 + isdigit 校验
        print("请输入正整数")
        return
    primes = find_primes(int(raw))
    print(f"2~{raw} 之间共 {len(primes)} 个素数:")
    for i, p in enumerate(primes, start=1):  # enumerate 带编号
        print(f"{p:>4}", end="\n" if i % 10 == 0 else " ")
    if len(primes) % 10 != 0:
        print()

def menu_stats() -> None:
    text = input("请输入逗号分隔的数字: ")
    nums = parse_numbers(text)
    if not nums:                       # 空列表为假 -> 提示
        print("输入无效(需要数字,如: 1,2.5,-3)")
        return
    total = sum(nums)                  # 内建聚合函数
    print(f"个数: {len(nums)} | 总和: {total:.2f} "
          f"| 平均: {total / len(nums):.2f} "
          f"| 最大: {max(nums)} | 最小: {min(nums)}")
```

```
for i, v in enumerate(nums[:3]):    # 前三个元素的下标与值
    print(f" 第{i + 1}个元素 = {v}")
```

第二段:字符串分析与主菜单。match 分发替代 switch, case _ + continue 处理非法选项,while-else 收尾。

```
def menu_text() -> None:
    s = input("请输入一句话: ")
    alpha = digit = space = other = 0
    for ch in s:                    # 遍历字符串
        if ch.isalpha():
            alpha += 1
        elif ch.isdigit():
            digit += 1
        elif ch.isspace():
            space += 1
        else:
            other += 1
    print(f"字母 {alpha} | 数字 {digit} | 空格 {space} | 其他 {other}")
    for i, ch in enumerate(s):
        if not ch.isspace():
            print(f" 下标 {i}: {ch!r}")

def main() -> None:
    while True:                    # do-while 式主循环
        print("\n==== 工具箱 =====")
        print("1. 素数筛选    2. 数组统计")
        print("3. 字符串分析  0. 退出")
        choice = input("请选择: ").strip()
        match choice:              # match 分发(match 是语句)
            case "1":
                menu_sieve()
            case "2":
                menu_stats()
            case "3":
                menu_text()
            case "0":
                break                # 唯一出口
            case _:
                # 通配兜底
                print("无效选项,请重新输入")
                continue
    else:
        # while-else:循环没被 break 打断才执行;
        # 本程序 break 必被触发,else 不执行(演示语法)
        print("感谢使用,再见!")

if __name__ == "__main__":
    main()
```

设计说明:① 每个菜单功能独立成函数,主循环只做分发——这是"函数即模块"思想的雏形(第4章深入);② match 分发清晰表达意图,非法选项走 case _ 通配 + continue ;③ 解析函数对非法输入返回空列表,调用方用真值判定

拦截,数据流简单;④ 素数输出用 `enumerate` 的编号判断换行,避免维护独立计数变量;⑤ 注意 `match` 是语句不能"返回"值,所以分发函数各自 `return`,主循环只管调用——这是 Python 流程控制组织的通用模式。

下一章预告:第4章 函数与方法——"传对象引用"的传参语义、默认参数可变对象的经典坑、`*args/**kwargs`、闭包与晚期绑定、`lambda` 与高阶函数,函数是 Python 世界的第一公民。